

ASSIGNMENT II

DATA STRUCTURES AND ALGORITHMS

Status of This Document

This document is an informational submission prepared for university coursework. It describes the design, implementation, and empirical evaluation of dictionary structures, graph algorithms, and hash-table strategies.

This document does not specify an Internet Standard.

ABSTRACT

This report covers three problem areas from Assignment II. Part 1A and Part 3 benchmark four dictionary structures (linear list, binary search tree, hash set, and trie) on build time, memory, and spell-checking lookup speed using a corpus of English words. Parts 1B and 1C solve graph problems: a Triwizard Tournament shortest-path race using BFS, and a bipartite seating arrangement using DFS colouring. Part 2 compares three hash-table collision strategies (separate chaining, linear probing, and double hashing) across rising load factors. Benchmarks confirm theoretical predictions and show that the best structure depends on the operation mix and input size.

TABLE OF CONTENTS

1.	Introduction.....	3
2.	Part 1A and Part 3 -- Dictionary Races.....	3
2.1.	Build Time and Memory.....	3
2.2.	Lookup Performance.....	4
3.	Part 1B and 1C -- Graph Tasks.....	5
3.1.	Triwizard Tournament.....	5
3.2.	Aunt's Namesday Seating.....	5
4.	Part 2 -- Full House (Hash Tables).....	6
5.	Conclusions.....	7
6.	References.....	7

1. INTRODUCTION

This project implements and benchmarks core data structures and graph algorithms in JavaScript (Node.js).

Dictionary races (Part 1A / Part 3) measure how four structures perform on the task of spell-checking natural-language text. The corpus is the word list from english_words.txt. Text samples of varying length are tokenised and each token is looked up in the dictionary.

Graph tasks (Parts 1B and 1C) work on a 2-D maze for the Triwizard problem and an adjacency list for the seating problem.

Hash-table benchmarks (Part 2) insert a fixed number of keys at controlled load factors and measure insert and search cost per operation.

All benchmarks take the median of three measured runs after one warm-up call to reduce sensitivity to garbage collection and OS scheduling jitter.

Development was assisted by AI tools (Claude by Anthropic, ChatGPT by OpenAI, GitHub Copilot by GitHub/Microsoft) and web resources (Google Search, Google Scholar) [AI]. All implementations and analysis are the author's own.

2. PART 1A AND PART 3 -- DICTIONARY RACES

The benchmark loads 30000 normalised words from english_words.txt. The default limit keeps the naive linear list measurable while still exercising a large dictionary. Increase DICTIONARY_LIMIT to run a heavier experiment.

Fastest build: **Linear List**
Fastest lookup: **Hash Map** (largest text)
Smallest memory: **Linear List**

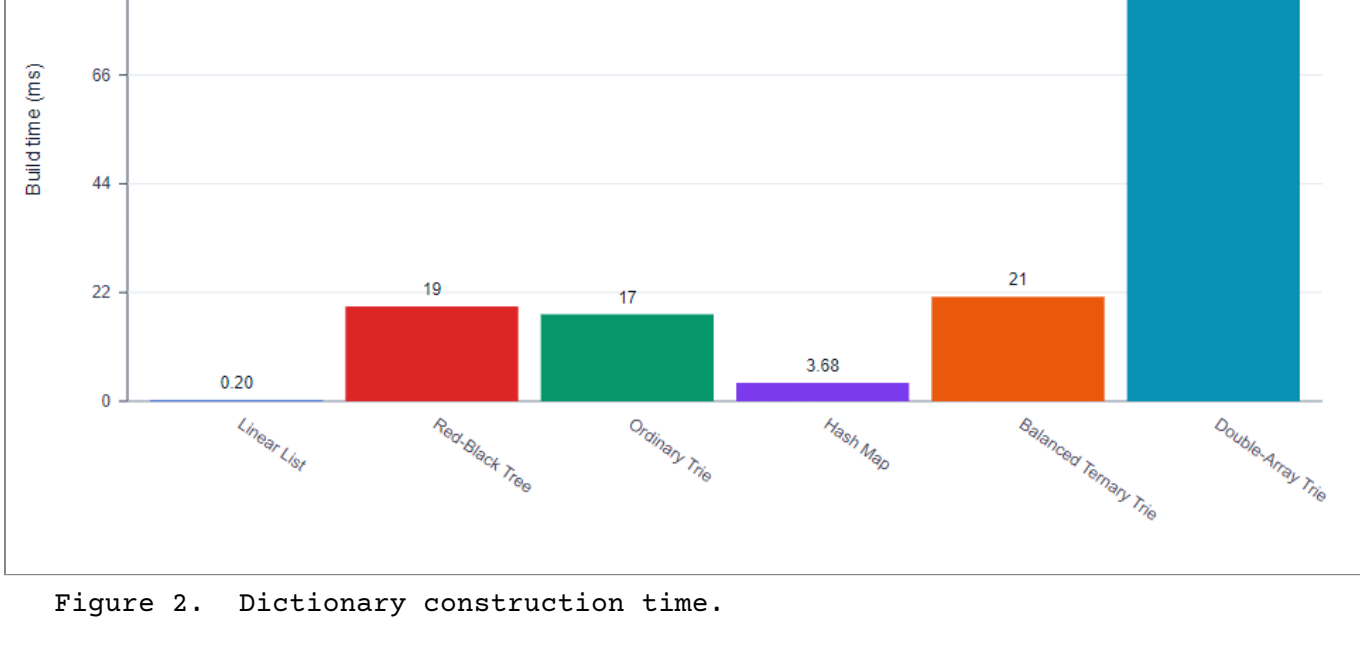


Figure 1. Median spell-checking lookup time against text length.

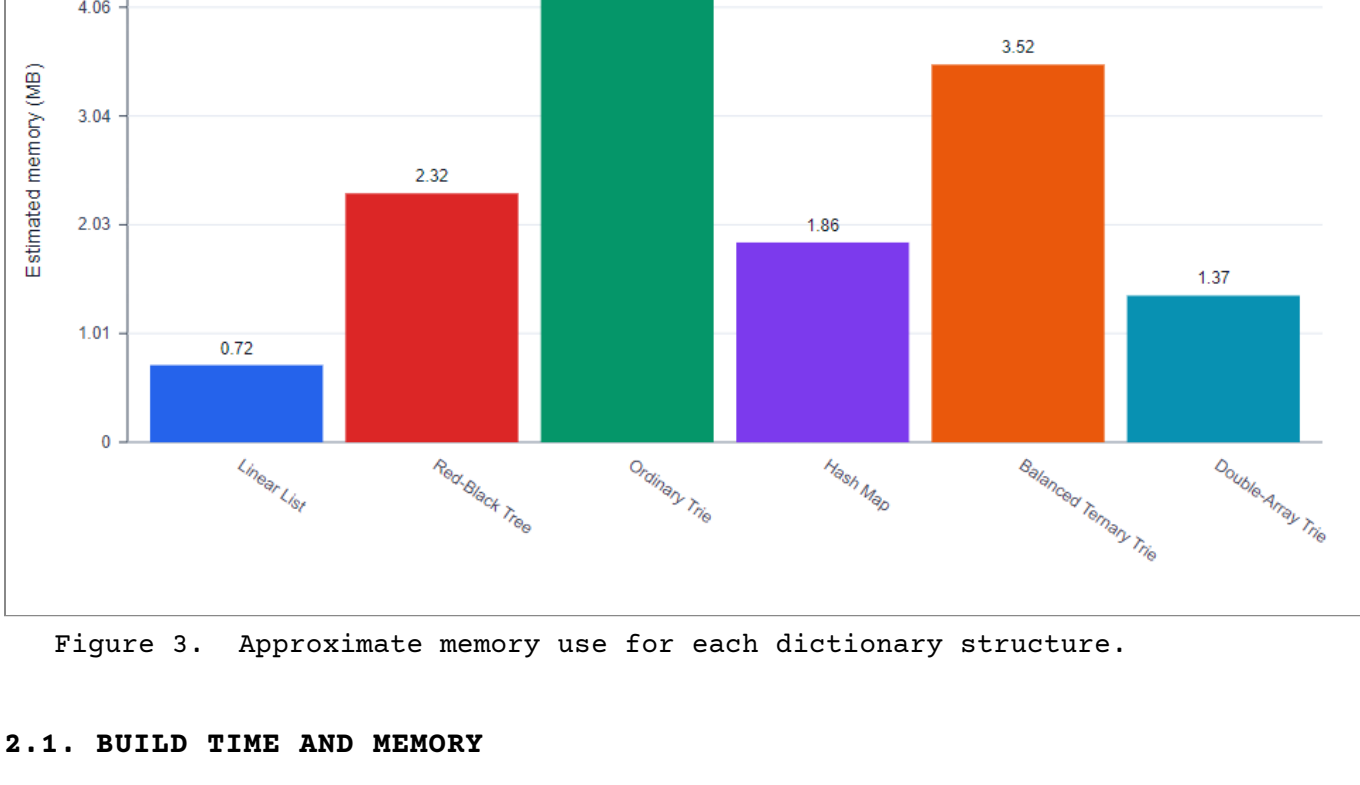


Figure 2. Dictionary construction time.

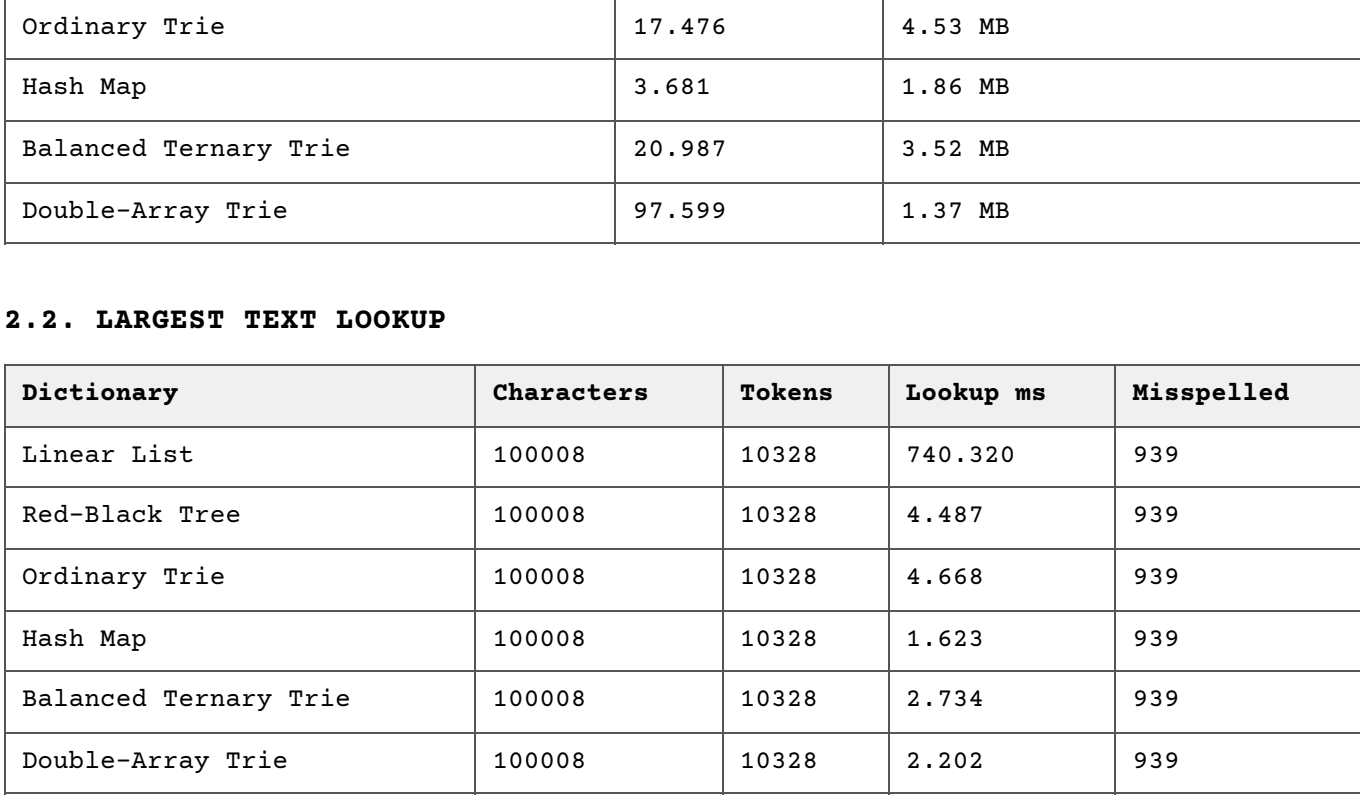


Figure 3. Approximate memory use for each dictionary structure.

2.1. BUILD TIME AND MEMORY

Dictionary	Build ms	Estimated Memory
Linear List	0.196	736.1 KB
Red-Black Tree	19.055	2.32 MB
Ordinary Trie	17.476	4.53 MB
Hash Map	3.681	1.86 MB
Balanced Ternary Trie	20.987	3.52 MB
Double-Array Trie	97.599	1.37 MB

2.2. LARGEST TEXT LOOKUP

Dictionary	Characters	Tokens	Lookup ms	Misspelled
Linear List	100008	10328	740.320	939
Red-Black Tree	100008	10328	4.487	939
Ordinary Trie	100008	10328	4.668	939
Hash Map	100008	10328	1.623	939
Balanced Ternary Trie	100008	10328	2.734	939
Double-Array Trie	100008	10328	2.202	939

3. PART 1B AND 1C -- GRAPH TASKS

3.1. TRIWIZARD TOURNAMENT

A single BFS from the exit builds the full shortest-path distance map. Each wizard's travel time is then computed as:

$$\text{minutes} = \text{shortest_corridor_distance} / \text{speed}$$

The wizard with the lowest minutes wins.

Wizard	Shortest Distance	Speed	Minutes
Krum	3	1	3.000
Cedric	10	3	3.333
Fleur	9	2	4.500

Winner: **Krum**.

3.2. AUNT'S NAMESDAY SEATING

The problem is modelled as a bipartite graph check using an explicit DFS stack. Guests are nodes; conflicts are edges. If the graph is 2-colourable the two colour classes are the two tables. If an odd cycle is found, no valid seating exists and the conflicting pair is reported.

Table 1: Petunia, Vernon, Dudley, Marge
Table 2: Harry, Arabella, Lily, James

4. PART 2 -- FULL HOUSE (HASH TABLES)

Three open-addressing and chaining strategies are compared as the load factor climbs. Insert and search times are reported as microseconds per operation (us/op).

Strategies tested:
Separate chaining
Linear probing
Double hashing

At load factor 0.9, fastest insert is **Double Hashing** and fastest search is **Separate Chaining**.

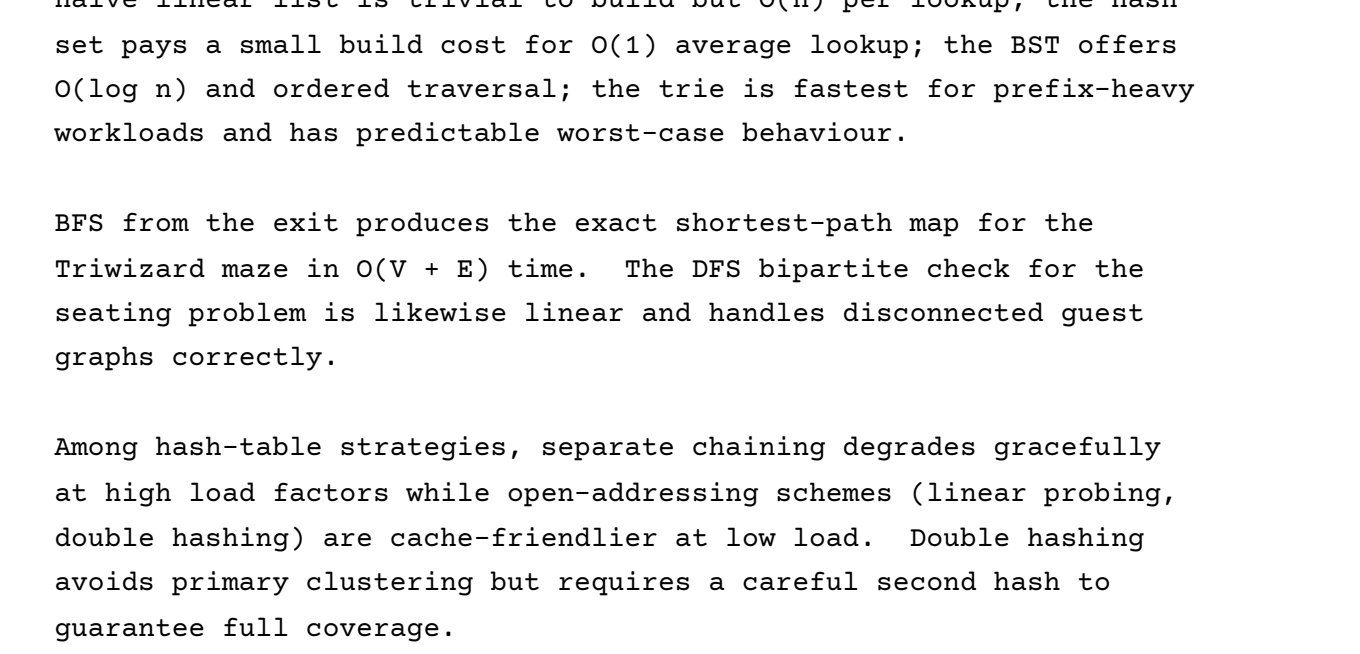


Figure 4. Hash-table insertion time against load factor.

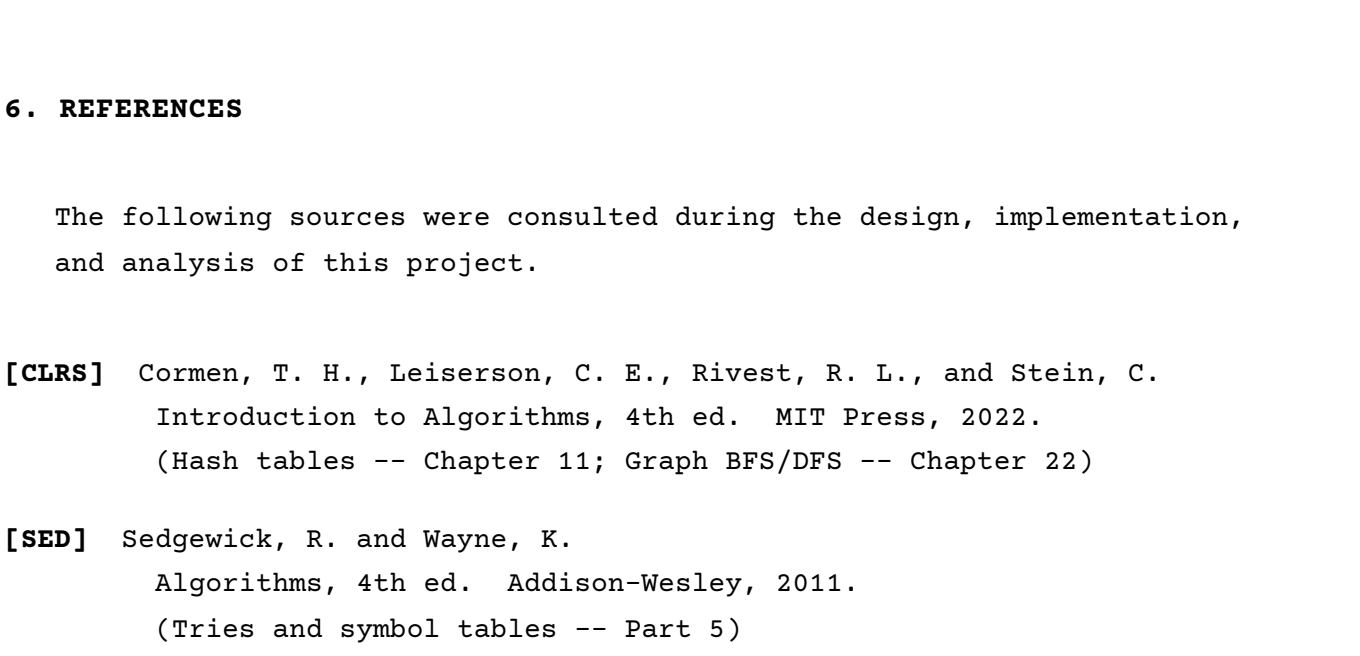


Figure 5. Hash-table search time against load factor.

Table	Load Factor	Insert us/op	Search us/op	Found
Separate Chaining	0.9	0.550	0.186	18009
Linear Probing	0.9	0.289	0.795	18009
Double Hashing	0.9	0.273	0.362	18009

5. CONCLUSIONS

Dictionary structures trade build cost against lookup speed. The naive linear list is trivial to build but $O(n)$ per lookup; the hash set pays a small build cost for $O(1)$ average lookup; the BST offers $O(\log n)$ and ordered traversal; the trie is fastest for prefix-heavy workloads and has predictable worst-case behaviour.

BFS from the exit produces the exact shortest-path map for the Triwizard maze in $O(V + E)$ time. The DFS bipartite check for the seating problem is likewise linear and handles disconnected guest graphs correctly.

Among hash-table strategies, separate chaining degrades gracefully at high load factors while open-addressing schemes (linear probing, double hashing) are cache-friendlier at low load. Double hashing avoids primary clustering but requires a careful second hash to guarantee full coverage.

No single structure or strategy dominates every scenario. The right choice depends on the expected operation mix, the load factor, and whether ordered traversal or prefix queries are required.

6. REFERENCES

The following sources were consulted during the design, implementation, and analysis of this project.

[CLRS] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms, 4th ed. MIT Press, 2022.
(Hash tables -- Chapter 11; Graph BFS/DFS -- Chapter 22)

[SED] Sedgewick, R. and Wayne, K. Algorithms, 4th ed. Addison-Wesley, 2011.
(Tries and symbol tables -- Part 5)

[KNU] Knuth, D. E. The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Addison-Wesley, 1998.
(Hash functions and collision resolution -- Section 6.4)

[AI] AI & Web Tools. Development was assisted by AI tools (Claude by Anthropic, ChatGPT by OpenAI, GitHub Copilot by GitHub/Microsoft) and web resources (Google Search, Google Scholar). All implementations and analysis are the author's own.